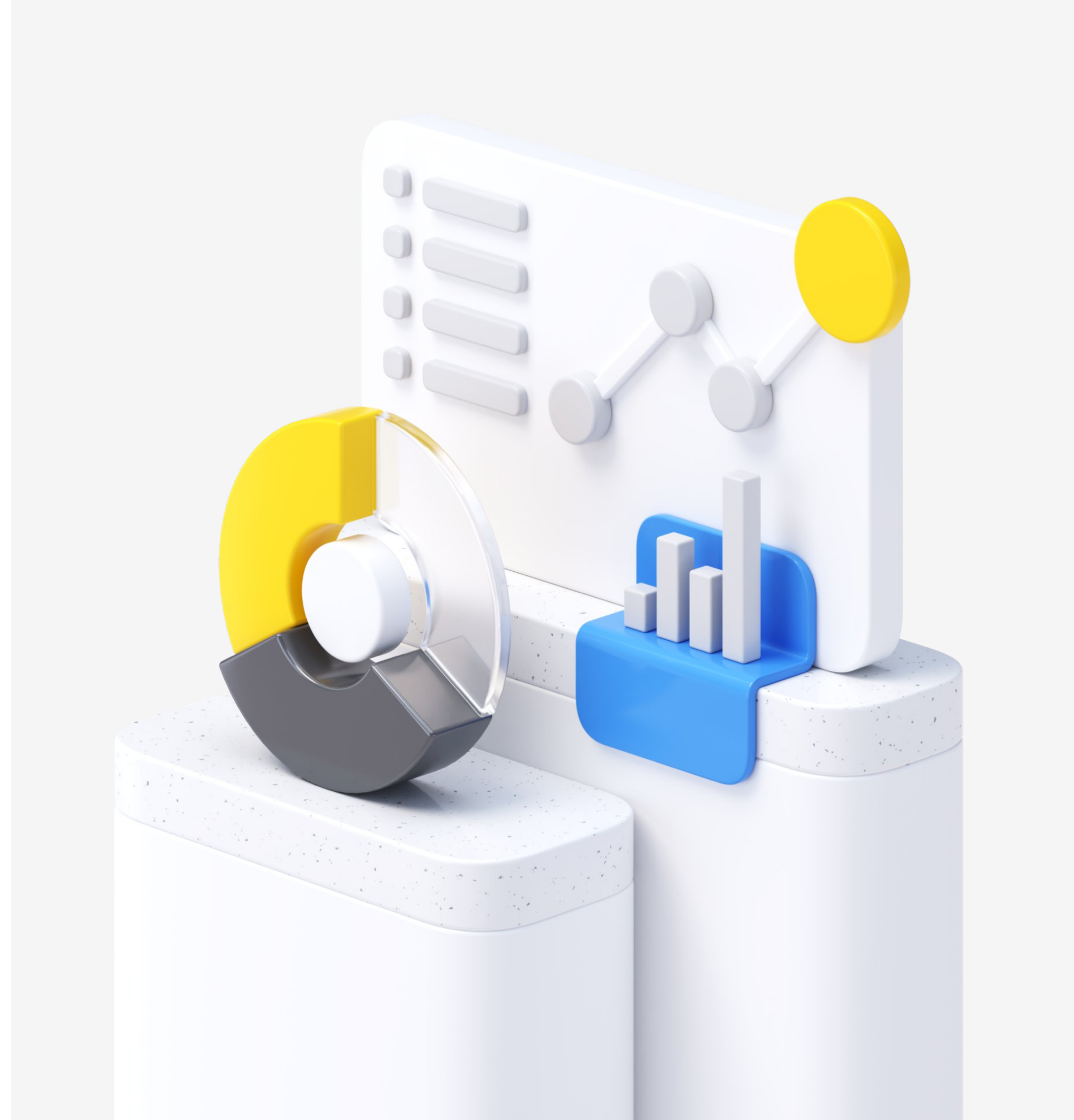




Лекция 2

Swift: переменные, типы, операторы, циклы

Анастасия, t.me/nasty_ru



План презентации

- Синтаксис Swift
- Типы данных
- Операторы
- Опционал
- Коллекции
- Циклы

План презентации

- Синтаксис Swift
- Типы данных
- Операторы
- Опционал
- Коллекции
- Циклы

Комментарии

Неисполняемый текст в коде, игнорируются компилятором Swift во время компиляции кода.

Однострочные

```
// это комментарий
```

Многострочные

```
/* это комментарий,  
написанный на двух строках */
```

Вложенные

```
/* это начало первого комментария  
/* это второй, вложенный */  
это конец первого комментария */
```


Константы

```
let constant = 10
```

Переменные

```
var variable = 10  
variable = 35
```

Как выбрать, что использовать, чтобы избежать случайных некорректных изменений значений:

- не знаете, какой вид переменной выбрать → let
- значение нужно будет изменить → var

Точка с запятой

Swift не требует писать
точку с запятой (;)

Однако вы можете делать это, если хотите

Но если вы хотите написать несколько
отдельных выражений на одной
строке:

```
let cat = "cat"; print(cat)
```

Вывод данных

```
print("Hello!")
```

Можно включить переменные и константы:

```
let person = "world"  
print("Hello, \(person)!")
```


Live coding

План презентации

- Синтаксис Swift
- **Типы данных**
- Операторы
- Опционал
- Коллекции
- Циклы

Базовые типы данных

Целые числа
Int

```
let number1: Int = 1  
let number2: Int = -1
```

Числа с плавающей
точкой
Float/Double

```
let pi: Double = 3.14
```

Строки
String

```
let name: String = "name"
```

Булевые
Bool

```
let correct: Bool = true  
let incorrect: Bool = false
```

И это не все...

Аннотация ТИПОВ

Swift - строго типизированный язык и если создается переменная одного типа, то другой тип в нее записать не получится.

Чтобы указать, какому типу будет соответствовать переменная, напишите его после двоеточия:

```
var message: String
```

Однако когда вы даете начальное значение на момент объявления, Swift сам может вывести тип и указывать его не обязательно:

```
var message = "Message" // тип String
```

Перечисления

Это ещё один вид типов в нашу копилку типов.

```
enum Figure {  
    case square  
    case circle  
    case rectangle  
}
```

Перечисление определяет общий тип для группы связанных значений, которые ограничивают эти возможные значения.

Перечисления

Можно задать ассоциативные значения

```
enum Figure {  
    case square(Int)  
    case circle(Int)  
    case rectangle(Int, Int)  
}
```

Ассоциативные значения могут быть любого типа, и типы значений могут отличаться для каждого члена перечисления

Live coding

План презентации

- Синтаксис Swift
- Типы данных
- **Операторы**
- Опционал
- Коллекции
- Циклы

Математические операторы

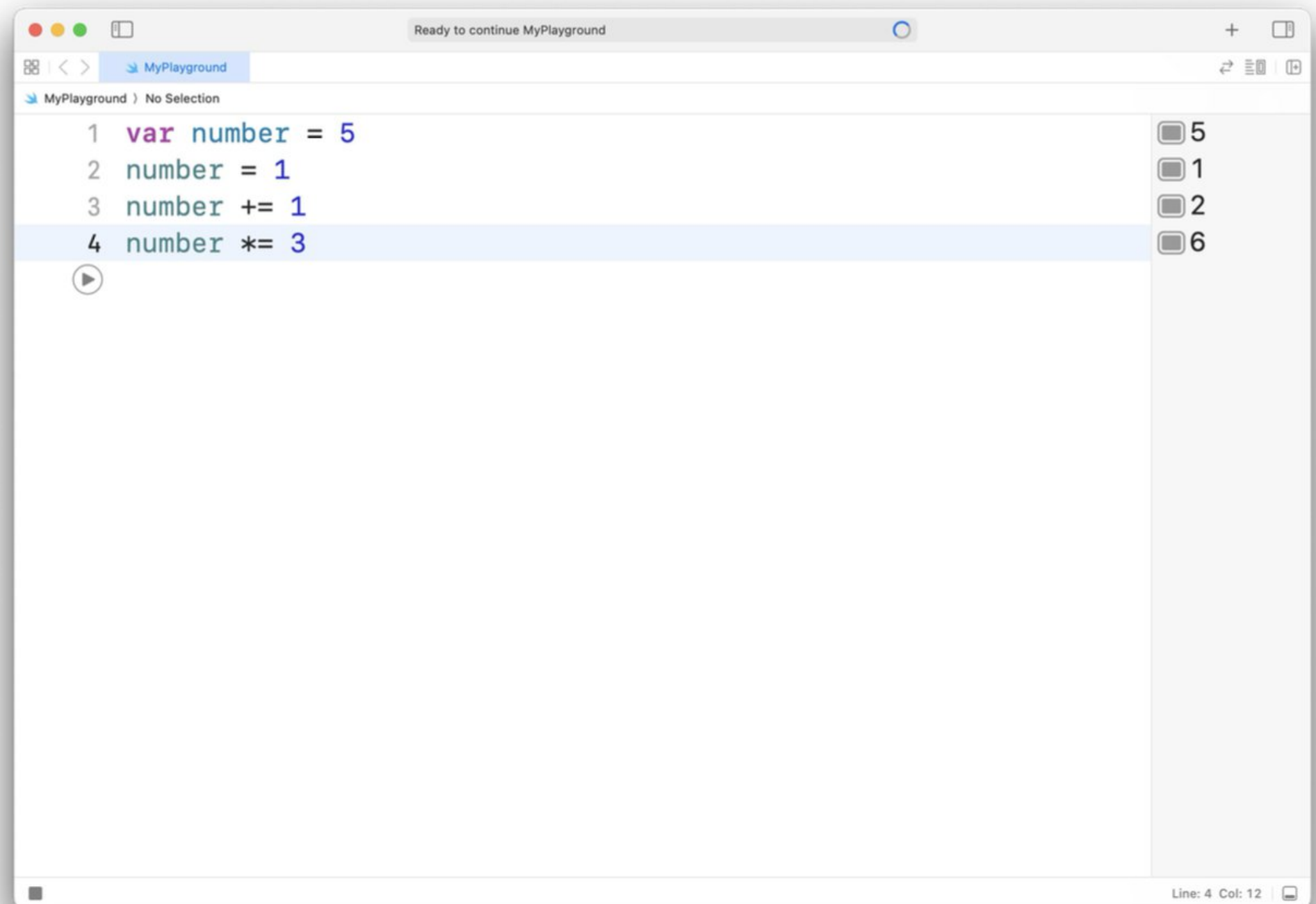
+ Сложение

- Вычитание

* Умножение

/ Деление

% Остаток от деления

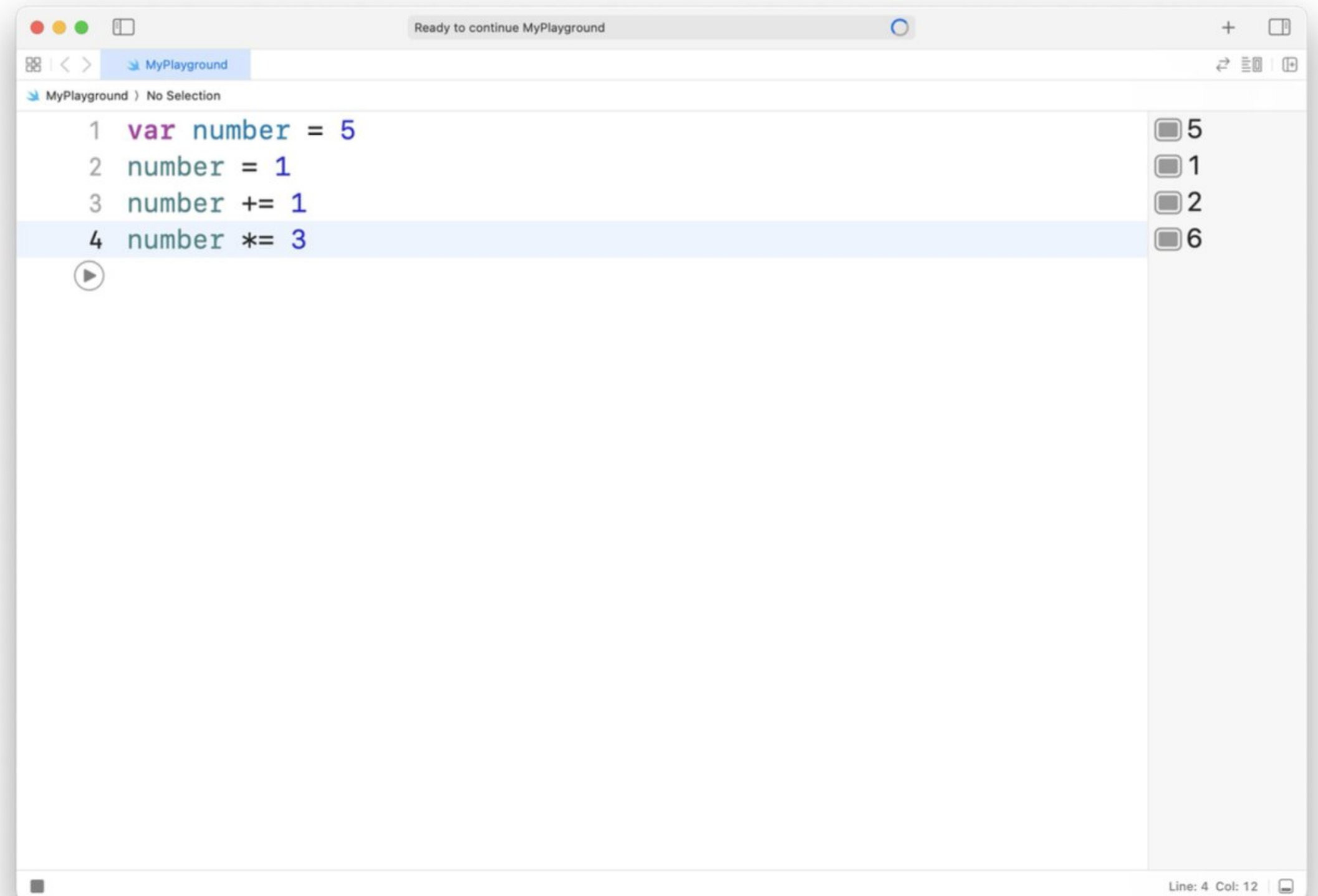


Операторы присваивания

= Оператор присваивания

Составные операторы присваивания:

+=, -=, *=, /=, %=



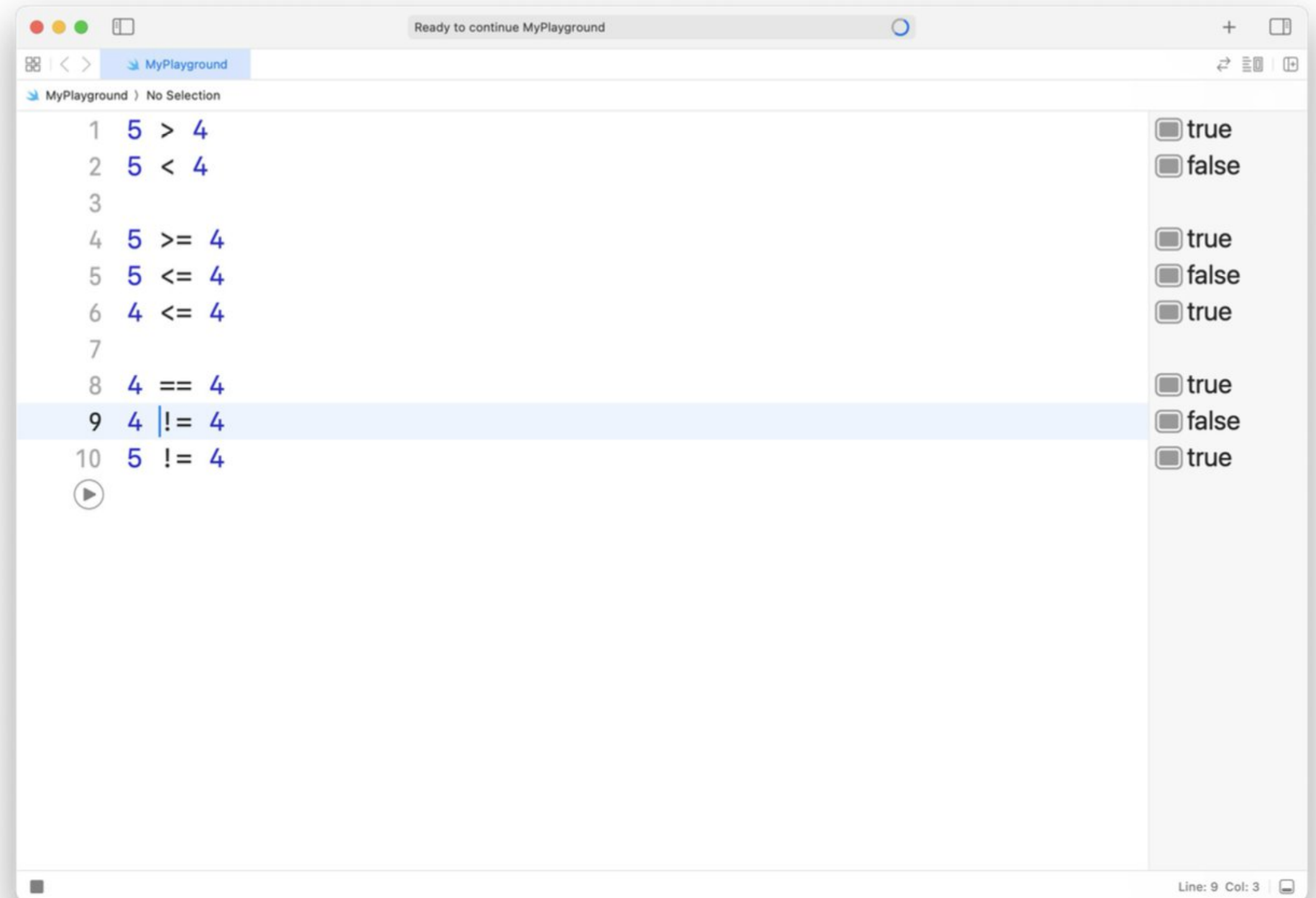
Операторы сравнения

>, < Больше, меньше

>=, <= Больше либо равно, меньше либо равно

== Равенство (равно)

!= Неравенство (неравно)

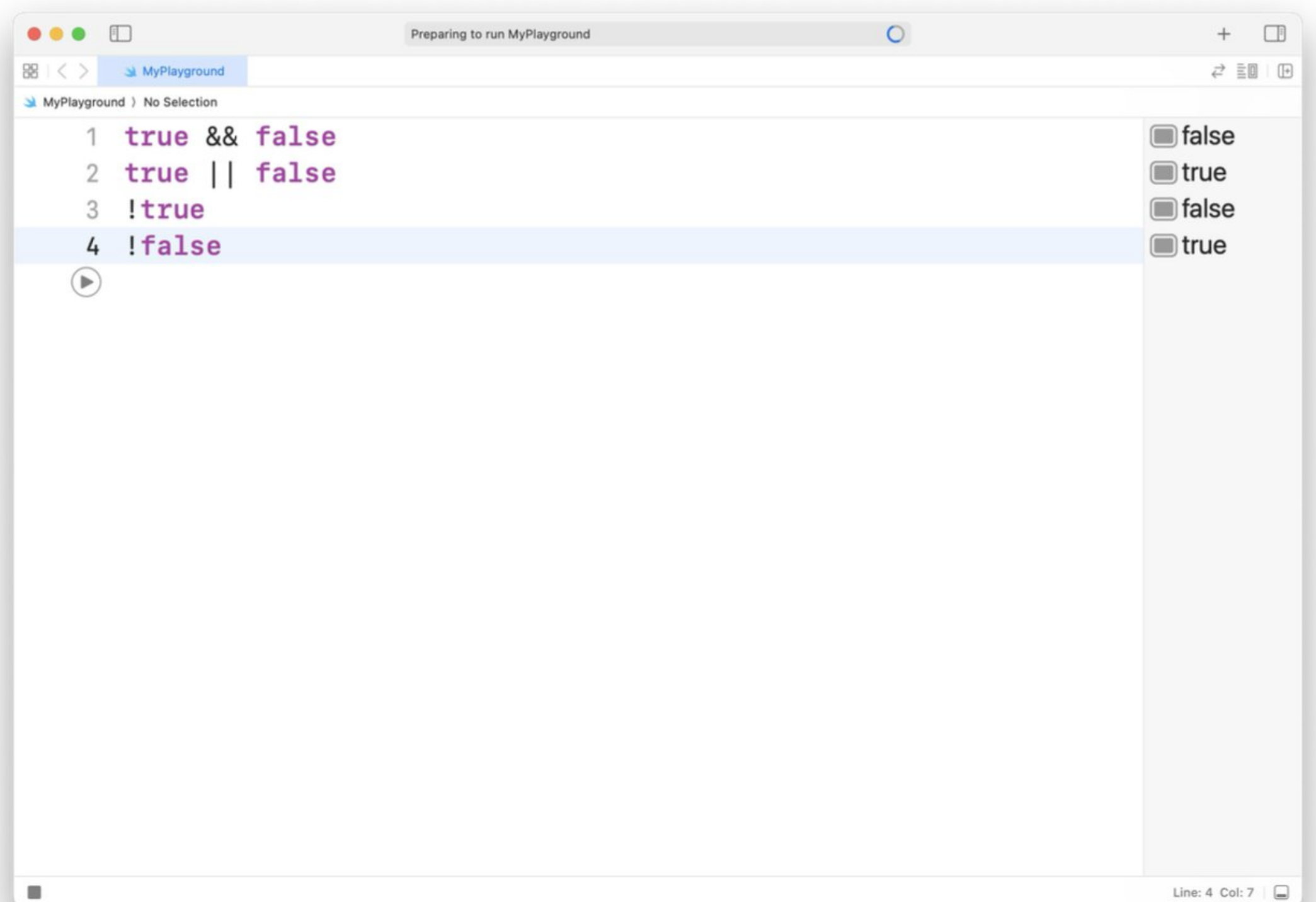


Логические операторы

&& Логическое "И"

|| Логическое "ИЛИ"

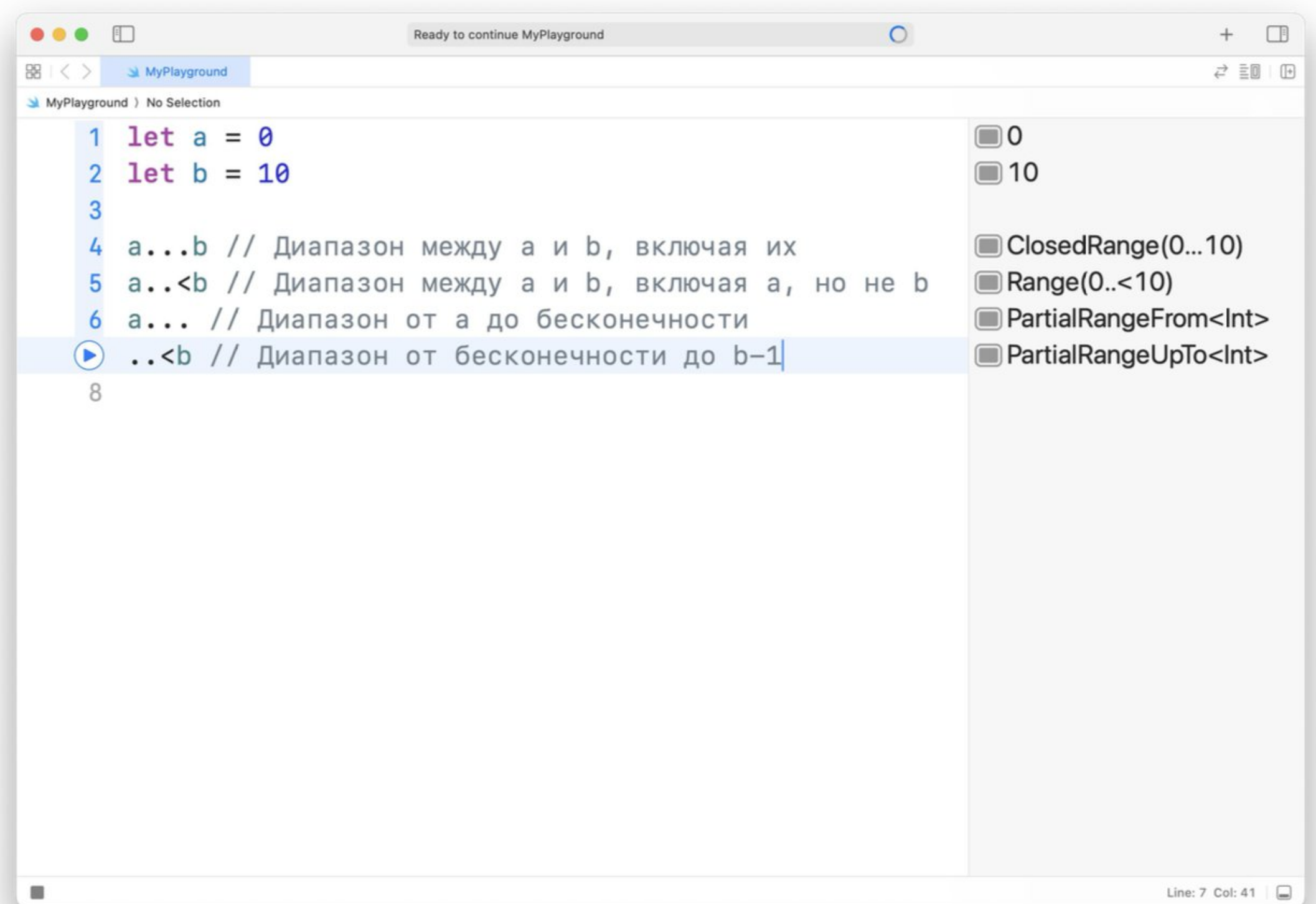
! Логическое отрицание "НЕ"



Операторы диапазона

Оператор может быть:

- замкнутым — то есть включать в себя границы
- полузамкнутым — исключать одну границу
- односторонним — не ограниченным с одной стороны

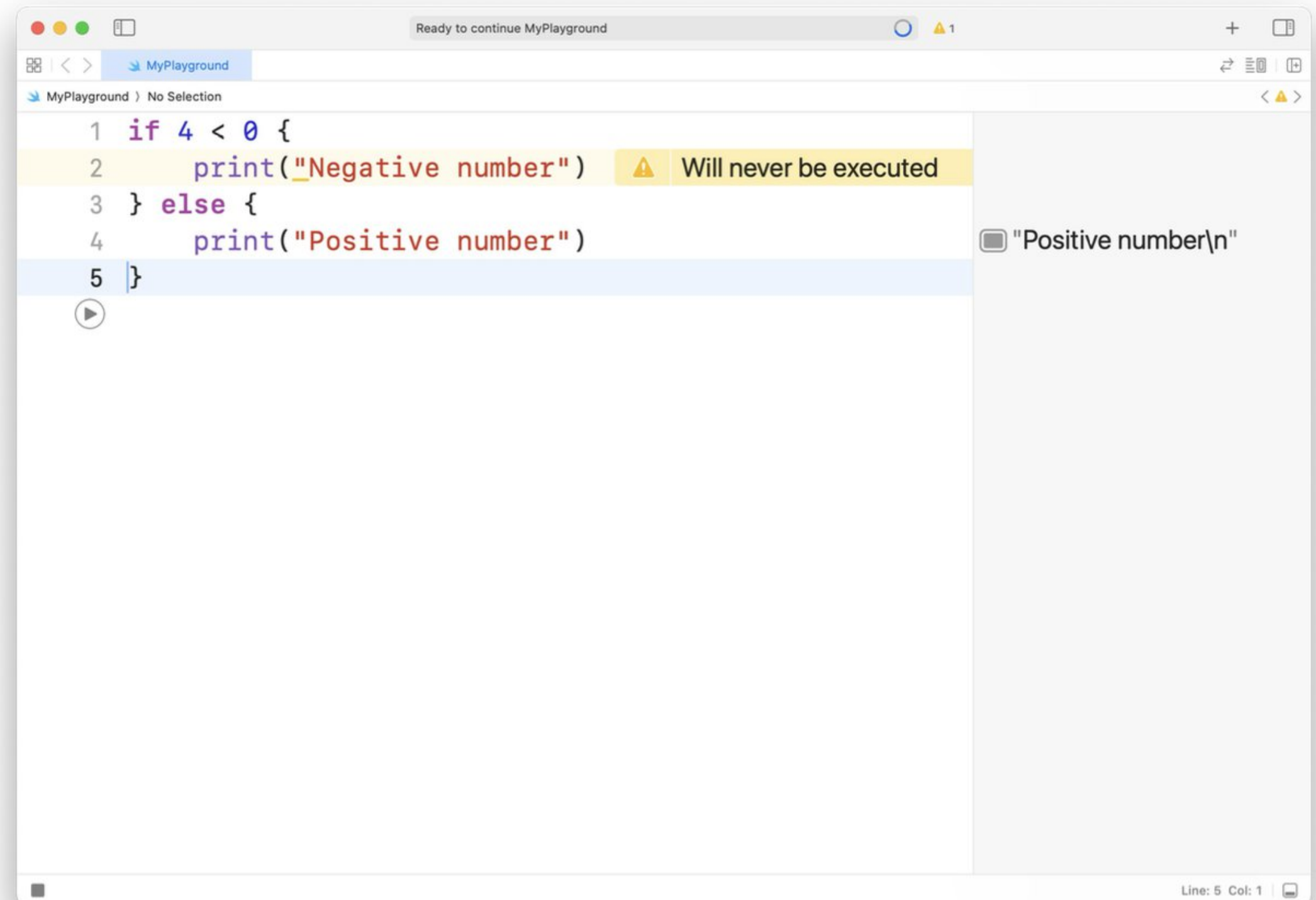


Условный оператор if

```
if <#Условие#> {  
    // Тело оператора при выполнении условия  
} else {  
    // Тело оператора при не выполнении условия  
}
```

Вместо условия мы можем подставить:

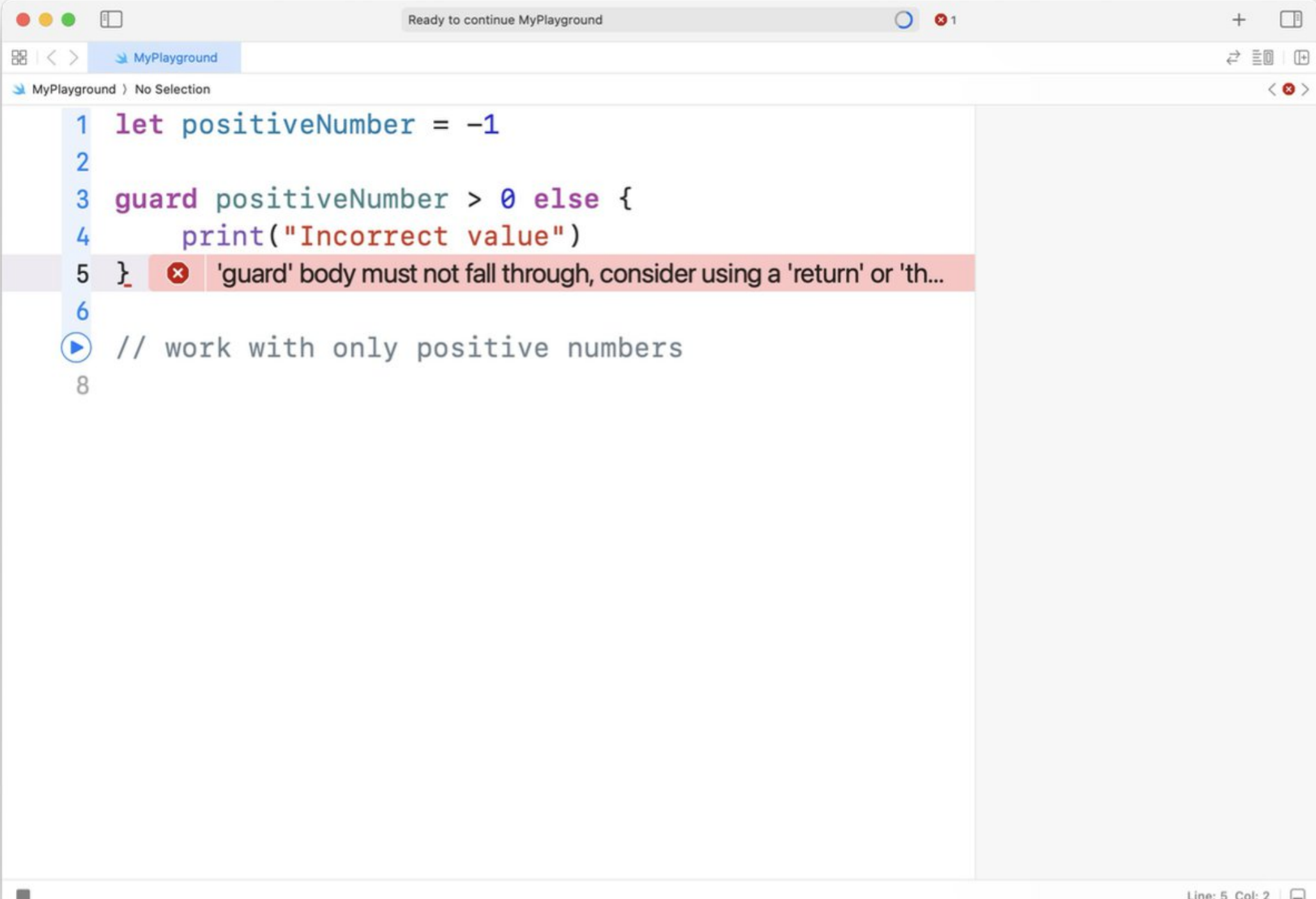
- булево значение
- какое-то выражение, результатом выполнения которого будет являться булево значение



Оператор guard

Инструкция guard используется для перевода контроля программы из области видимости, если одно или несколько условий не выполняются.

```
guard <#Условие#> else {  
    // Тело для исполнения, если условие false  
}
```



The screenshot shows a Swift Playground window titled "MyPlayground". The code in the editor is as follows:

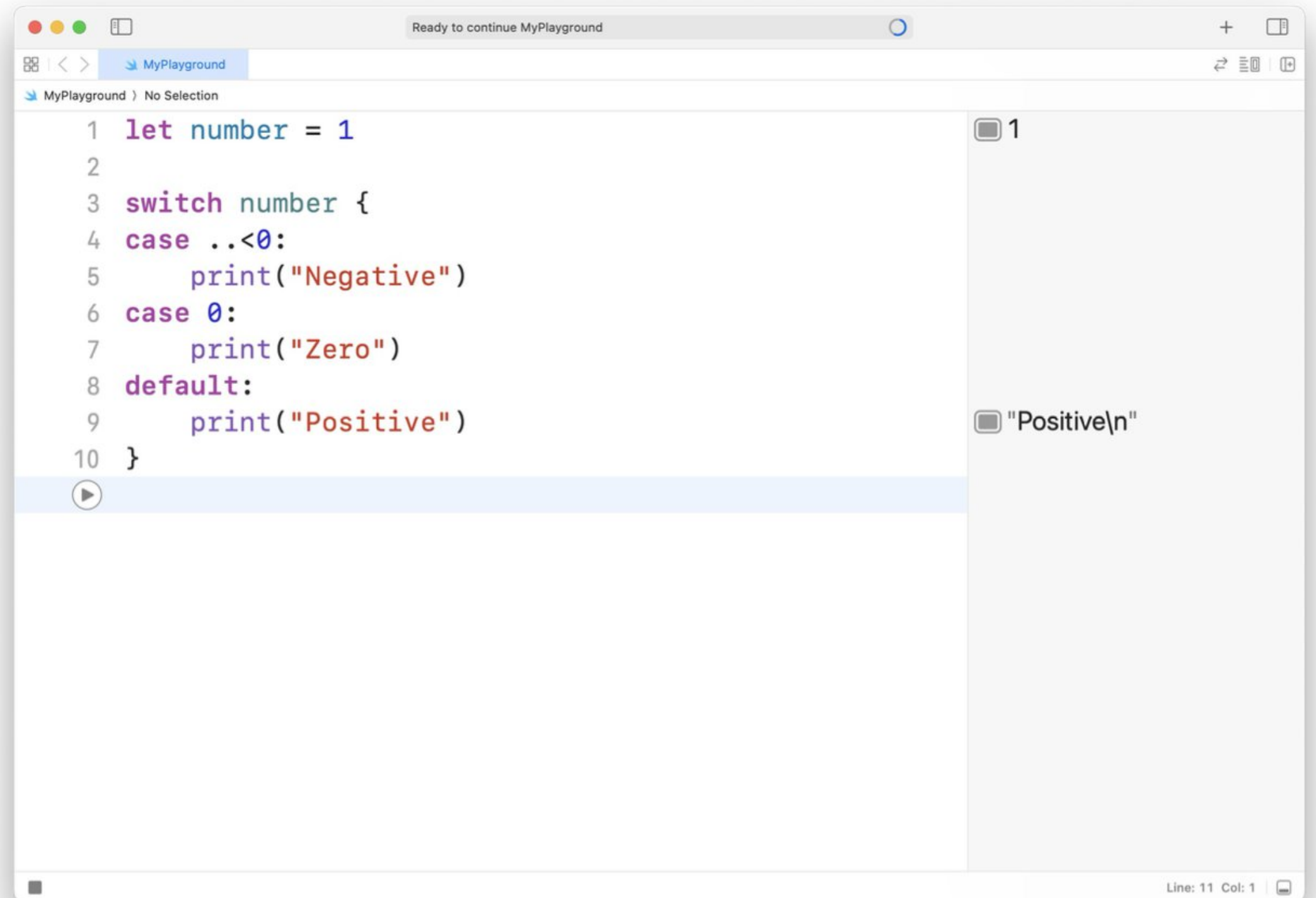
```
1 let positiveNumber = -1  
2  
3 guard positiveNumber > 0 else {  
4     print("Incorrect value")  
5 }  
6  
7 // work with only positive numbers  
8
```

A red error message is displayed on line 5: "'guard' body must not fall through, consider using a 'return' or 'th...'. The status bar at the bottom right indicates "Line: 5 Col: 2".

Оператор switch

Производит сравнение какого-то значения с возможными шаблонными. Если какое-либо значение совпало, то выполняется код, соответствующий этому шаблону:

```
switch некоторое значение {  
  case возможное значение:  
    действие 1  
  case возможное значение 2:  
    действие 2  
  default:  
    действие 3  
}
```



The screenshot shows a Swift Playground window titled "Ready to continue MyPlayground". The code editor contains the following Swift code:

```
1 let number = 1  
2  
3 switch number {  
4 case ..<0:  
5     print("Negative")  
6 case 0:  
7     print("Zero")  
8 default:  
9     print("Positive")  
10 }
```

The code is executed, and the output console on the right shows the results of the print statements:

```
1  
"Positive\n"
```

The status bar at the bottom right indicates "Line: 11 Col: 1".

Оператор switch & enum

```
let ourFigure: Figure = .circle
switch ourFigure {
case .square:
    print("It's square")
case .circle:
    print("It's circle")
case .rectangle:
    print("It's rectangle")
}
```

```
switch ourFigure {
case .circle:
    print("No corners")
default:
    print("4 corners")
}
```


Live coding

План презентации

- Синтаксис Swift
- Типы данных
- Операторы
- **Опционал**
- Коллекции
- Циклы

Опциональные типы (Optional)

Опциональные типы используются в тех случаях, когда значение может отсутствовать.

```
var optional: Int? = 1  
// optional содержит реальное Int значение 1
```

```
optional = nil  
// optional теперь не содержит значения
```

Если объявить опциональную переменную без значения по умолчанию, то переменная автоматически установится в nil:

```
var optional: String?  
// optional автоматически установится в nil
```

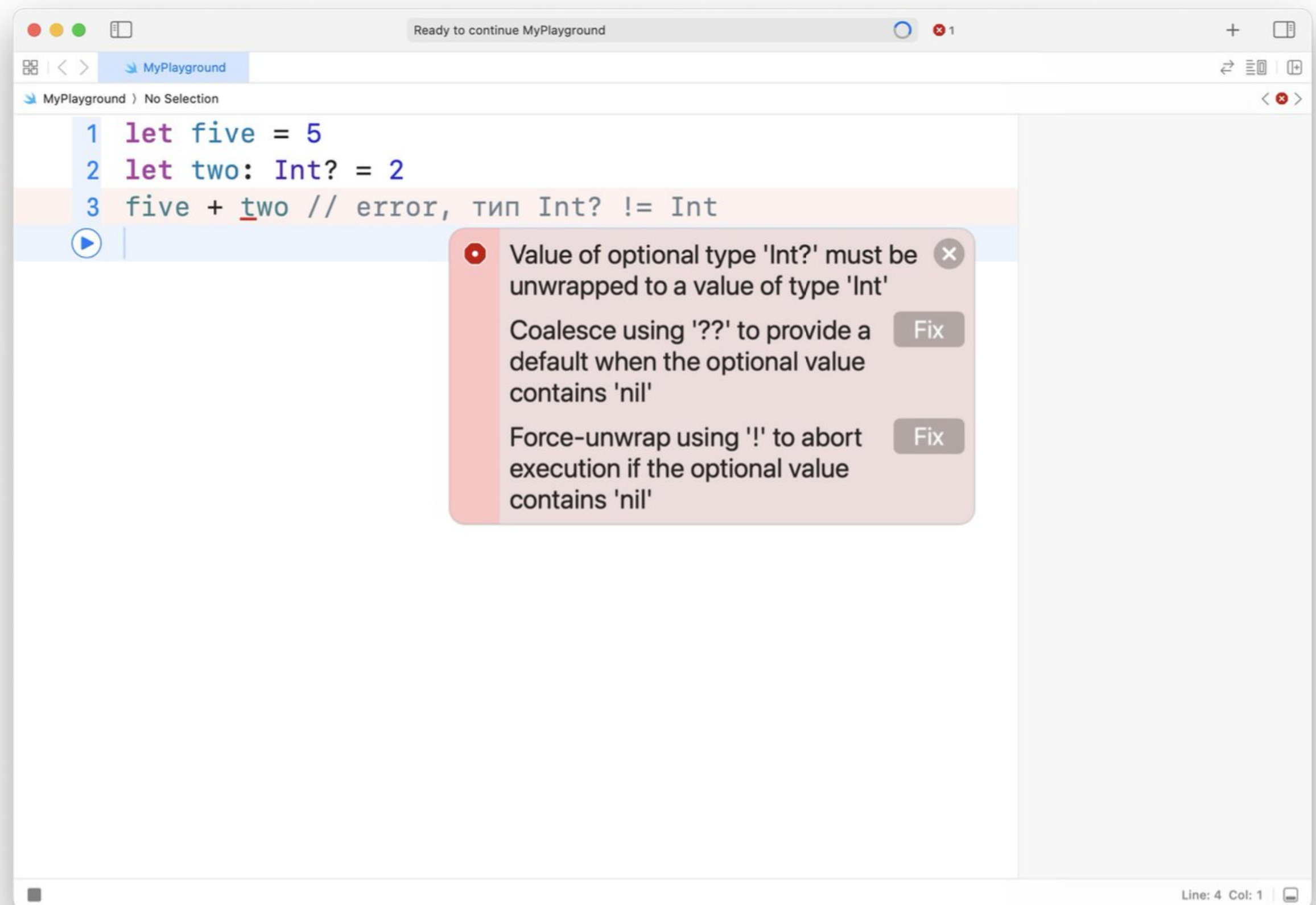
Опционал

Обратимся [к исходному коду Swift](#)

```
enum Optional<Wrapped> {  
    /// The absence of a value.  
    ///  
    /// In code, the absence of a value is typically written using the `nil`  
    /// literal rather than the explicit `.none` enumeration case.  
    case none  
    /// The presence of a value, stored as `Wrapped`.  
    case some(Wrapped)  
}
```

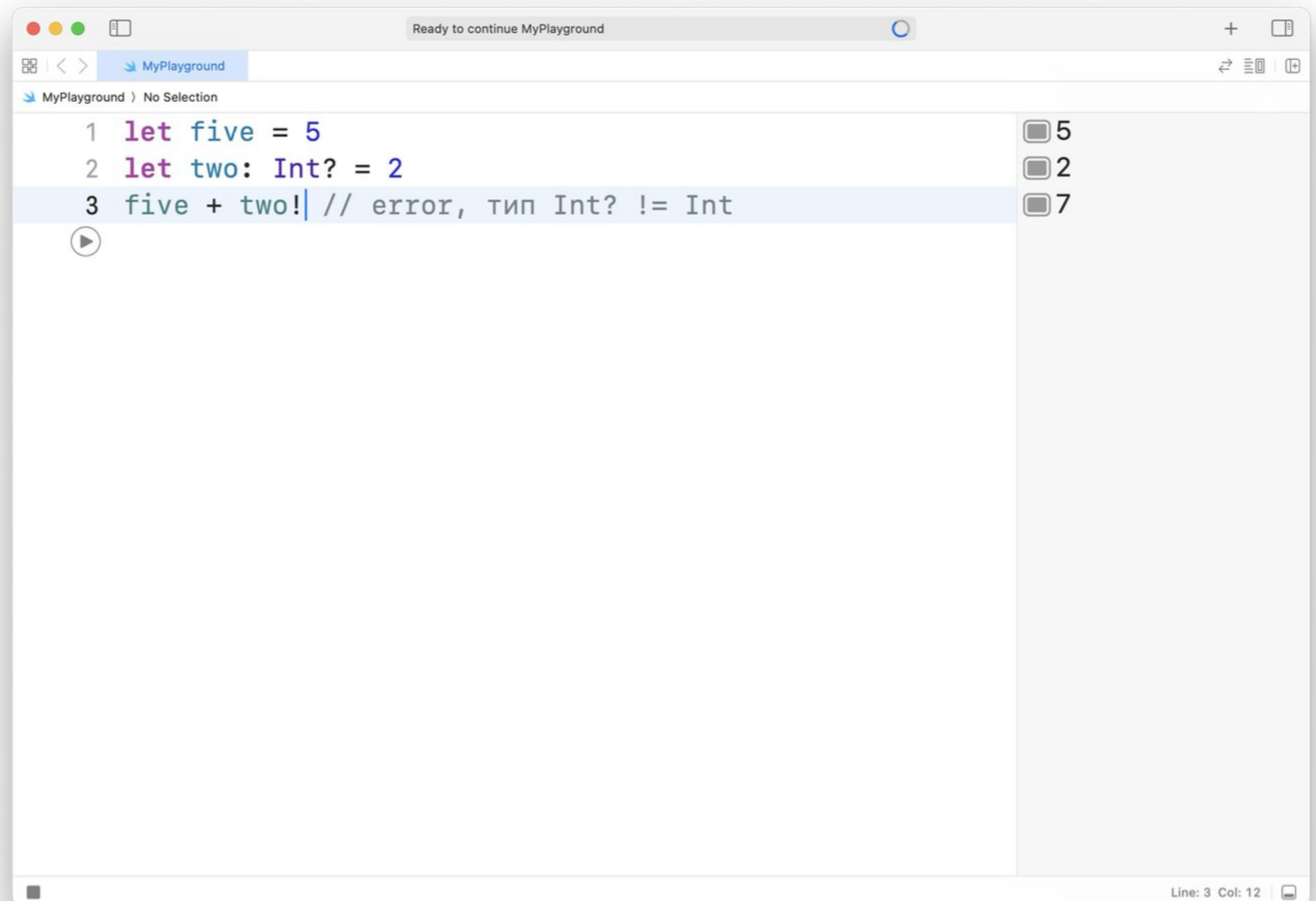

Работа с опционалами

Попробуем сложить Optional значение с обычным:



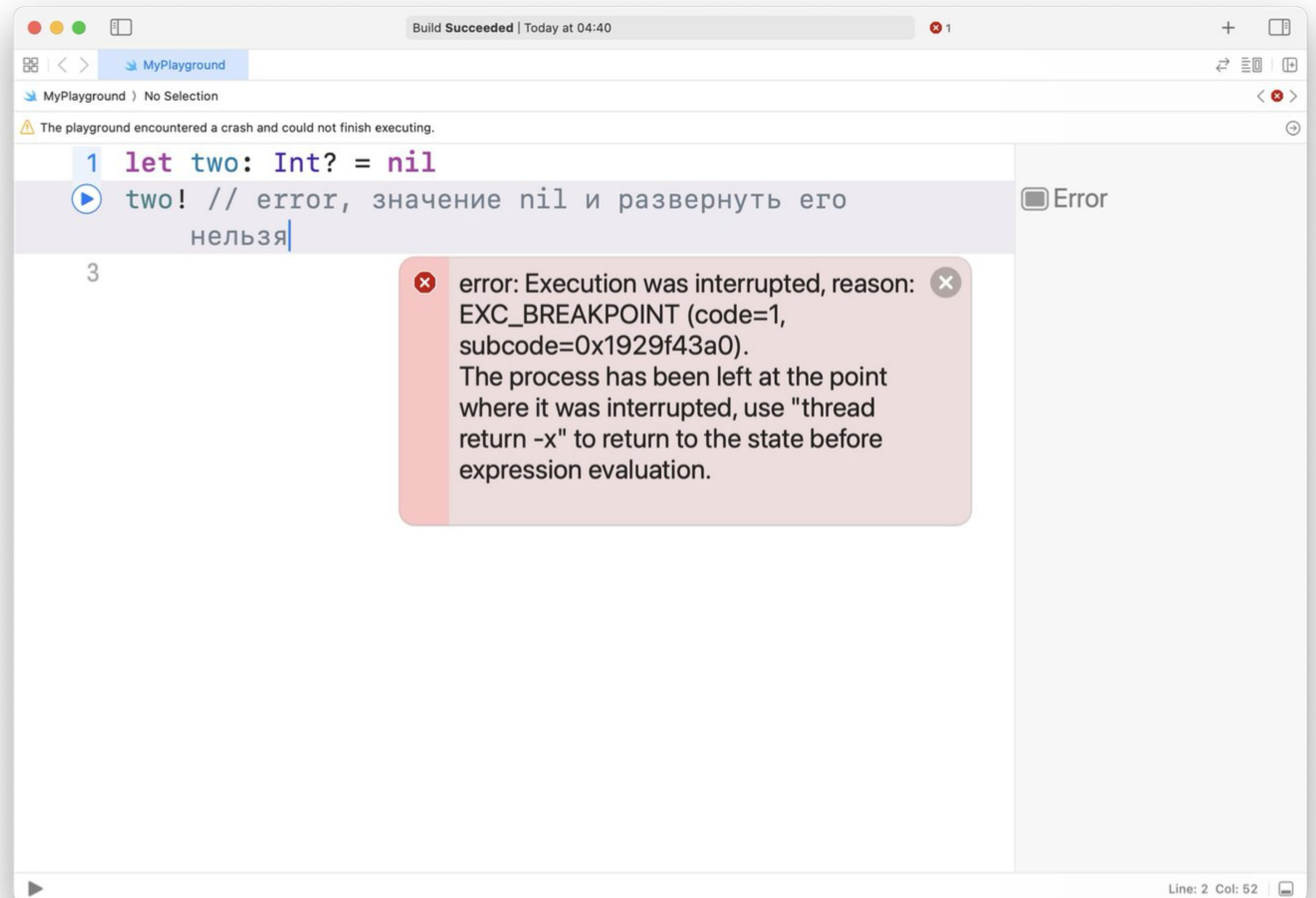
Работа с опционалами

Применим принудительное развертывание по подсказке компилятора:



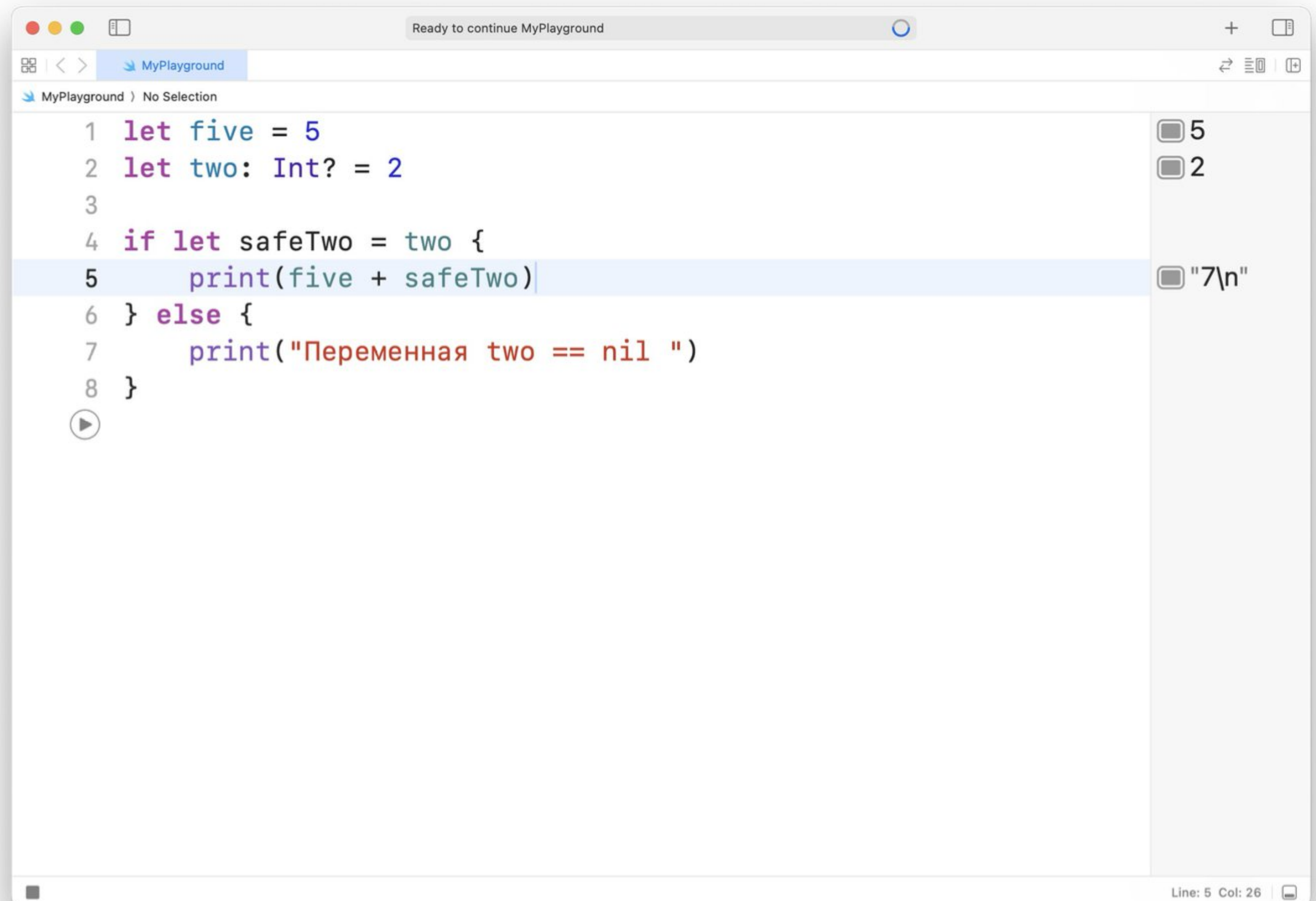
Работа с опционалами

Однако важно иметь в виду, что если в переменной лежит `nil`, то принудительное развертывание приведет к ошибке:



Работа с опционалами

Безопасное извлечение, даже если у опционала не оказалось значения, ваше приложение не прекратит свое выполнение.



```
1 let five = 5
2 let two: Int? = 2
3
4 if let safeTwo = two {
5     print(five + safeTwo)
6 } else {
7     print("Переменная two == nil ")
8 }

5
2
"7\n"
```

Line: 5 Col: 26

Live coding

План презентации

- Синтаксис Swift
- Типы данных
- Операторы
- Опционал
- **Коллекции**
- Циклы

Кортежи

Кортежи — набор значений, которые рассматриваются как один объект, нет необходимости, чтобы они были одного и того же типа.

Объявление кортежа:

```
var tupleName: (Int, String)
```

Действия с кортежами:

```
var student1: (String, Double, Int) = ("Настя", 3.4, 1)  
student1.2 += 1
```

```
var student2 = (name: "Никита", grade: 5.0, course: 2)  
let name2 = student2.name
```

Массивы

Массив — коллекция элементов **одного типа**, к которым можно обратиться по индексу - номеру элемента в массиве. **Индекс** число типа Int и начинается с нуля.

Объявление массива:

```
// Полная форма  
var имяМассива: Array<Тип>
```

```
// Краткая форма  
var имяМассива: [Тип]
```

Действия с массивами

Инициализация массива:

```
var numbers = [1, 2, 3, 4, 5]
```

Или пустой массив:

```
var numbers = [Int]()
```

```
var numbers2: [Int] = []
```


Действия с массивами

Размер массива:

```
numbers.count
```

`isEmpty` позволяет узнать, пуст ли массив:

```
numbers.isEmpty // вернет true/false
```

Доступ к элементам массива, каждый элемент в массиве

имеет определенный индекс, по которому мы можем

получить или изменить элемент:

```
print(numbers[0])
```

```
numbers[0] = 21
```

Действия с массивами

```
var numbers = [1, 2, 3]
```

Добавление элементов:

```
numbers.append(4)  
numbers += [5]  
numbers += [6, 7]  
numbers.insert(0, at: 0)
```

Удаление элементов:

```
numbers.remove(at: 0)  
numbers.remove(at: 10) // что будет?
```

Множества

Множество хранит различные значения одного типа в виде коллекции в **неупорядоченной** форме.

Тип должен быть **хэшируемым** (должен быть уникально идентифицируемым, предоставлять возможность для вычисления собственного значения хэша).

Все базовые типы (Int, String, Double, Bool) хэшируемы.

```
var alphabet = Set<String>( )
```

Действия с множествами

Объявление:

```
var alphabet: Set = ["a", "b", "c"]
```

Действия:

```
alphabet.insert("d")  
alphabet.remove("d")  
alphabet.contains("d")
```


Словари

Словарь — контейнер, который хранит несколько **значений одного и того же типа**. Каждое значение связано с уникальным **ключом**-идентификатором, те должен быть хэшируемым. В отличие от элементов в массиве, элементы в словаре не имеют определенного порядка.

```
// Полная форма  
var имяСловаря: Dictionary<Key, Value>  
  
// Краткая форма  
var имяСловаря: [Key: Value]
```

Действия со словарями

Объявление словаря:

```
var romeNumbers = ["I": 2, "V": 5, "X": 10]
```

Или пустой:

```
var numbers: [Int: String] = [:]
```

Доступ к элементам словаря:

```
romeNumbers["I"] = 1 // обновили
```

```
romeNumbers["L"] = 50 // добавили
```

```
romeNumbers["L"] = nil // удалили
```

Действия со словарями

Доступ к элементам словаря:

```
if let oldValue = romeNumbers.updateValue(50, forKey: "L") {  
    print("Старое значение \(oldValue).")  
}
```

```
if let number = romeNumbers["L"] {  
    print("L - \(number).")  
} else {  
    print("L не существует")  
}
```

```
if let removedValue = romeNumbers.removeValue(forKey: "L") {  
    print("Удалили \(removedValue).")  
} else {  
    print("L не существует")  
}
```

Сложность операций

	Массив	Словарь	Множество
Поиск элемента/проверка наличия	$O(n)$ по индексу $O(1)$	$O(1)$	$O(1)$
Удаление	$O(n)$	$O(1)$	$O(1)$
Вставка	$O(n)$	$O(1)$	$O(1)$

План презентации

- Синтаксис Swift
- Типы данных
- Операторы
- Опционал
- Коллекции
- **Циклы**

Цикл

Цикл решает проблему необходимости повторять действия без копирования одного и того же кода множество раз.

Блок кода, который нужно повторять, называется телом цикла. Условие определяет как долго повторять действие: можно задать четкое количество повторений или заставить цикл выполняться до достижения какого-то условия.



Циклы с заранее
известным числом
повторений



Циклы с предусловиями

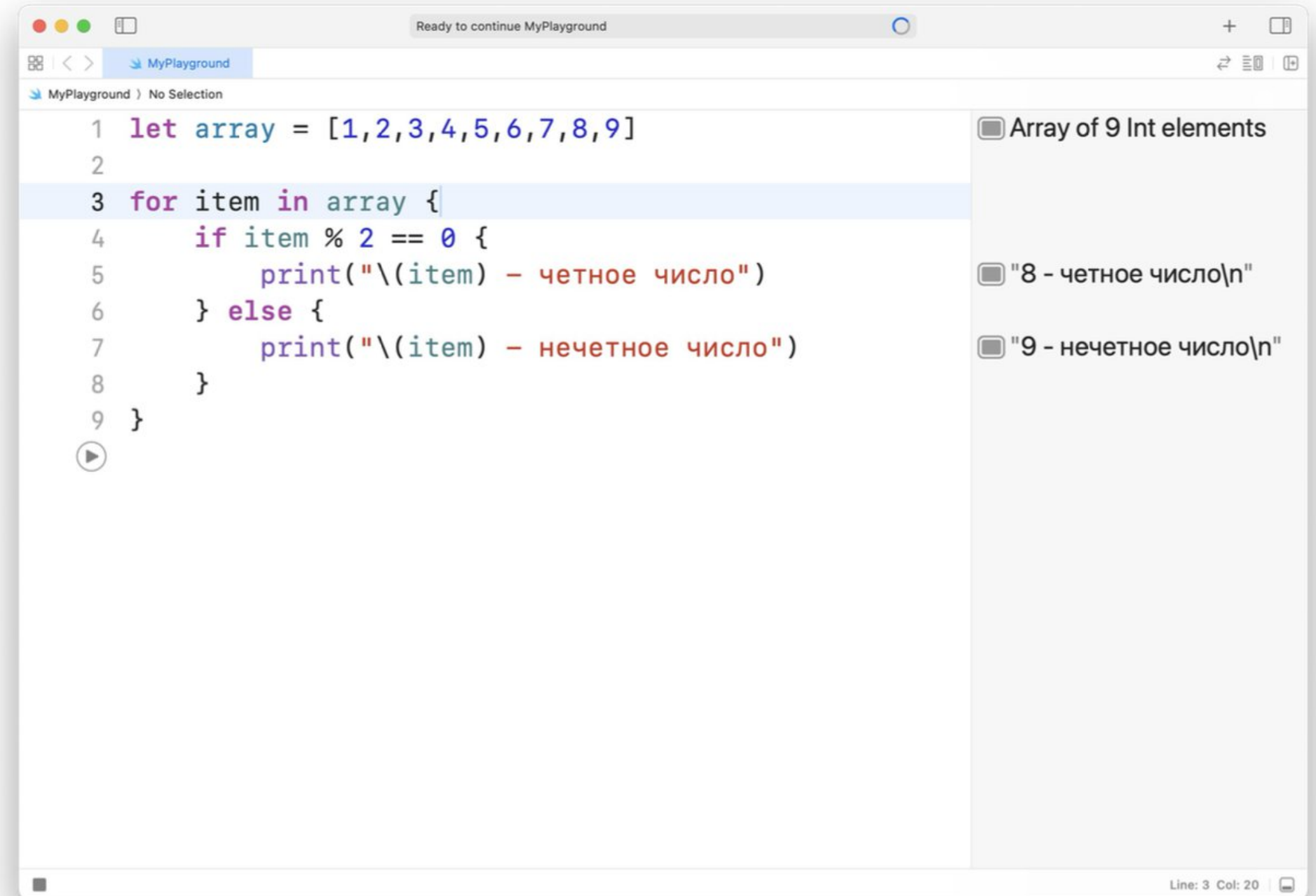


Циклы с постусловиями

Цикл for-in. Цикл с заранее известным числом повторений

С помощью цикла for-in мы можем перебрать элементы последовательности. Он имеет следующую форму:

```
for объект in последовательность {  
    // действия  
}
```



The screenshot shows a code editor window titled "MyPlayground" with the following Swift code:

```
1 let array = [1,2,3,4,5,6,7,8,9]  
2  
3 for item in array {  
4     if item % 2 == 0 {  
5         print("\(item) - четное число")  
6     } else {  
7         print("\(item) - нечетное число")  
8     }  
9 }
```

On the right side of the playground, there is a list of outputs:

- ☐ Array of 9 Int elements
- ☐ "8 - четное число\n"
- ☐ "9 - нечетное число\n"

The status bar at the bottom right indicates "Line: 3 Col: 20".

Цикл while. Цикл с предусловием

Оператор while проверяет некоторое условие, и если оно возвращает true, то выполняет блок кода.

Этот цикл имеет следующую форму:

```
while условие {  
    // тело цикла  
}
```



The screenshot shows a code editor window titled "MyPlayground" with a tab labeled "MyPlayground". The code is written in Swift and demonstrates a while loop. The code is as follows:

```
1 var i = 10  
2 while i > 0 { // пока значение i больше 0  
3     print(i) // 10, 9, 8, 7, 6, 5, 4, 3, 2, 1  
4     i-=1  
5 }
```

On the right side of the editor, there is a console output area showing the results of the code execution:

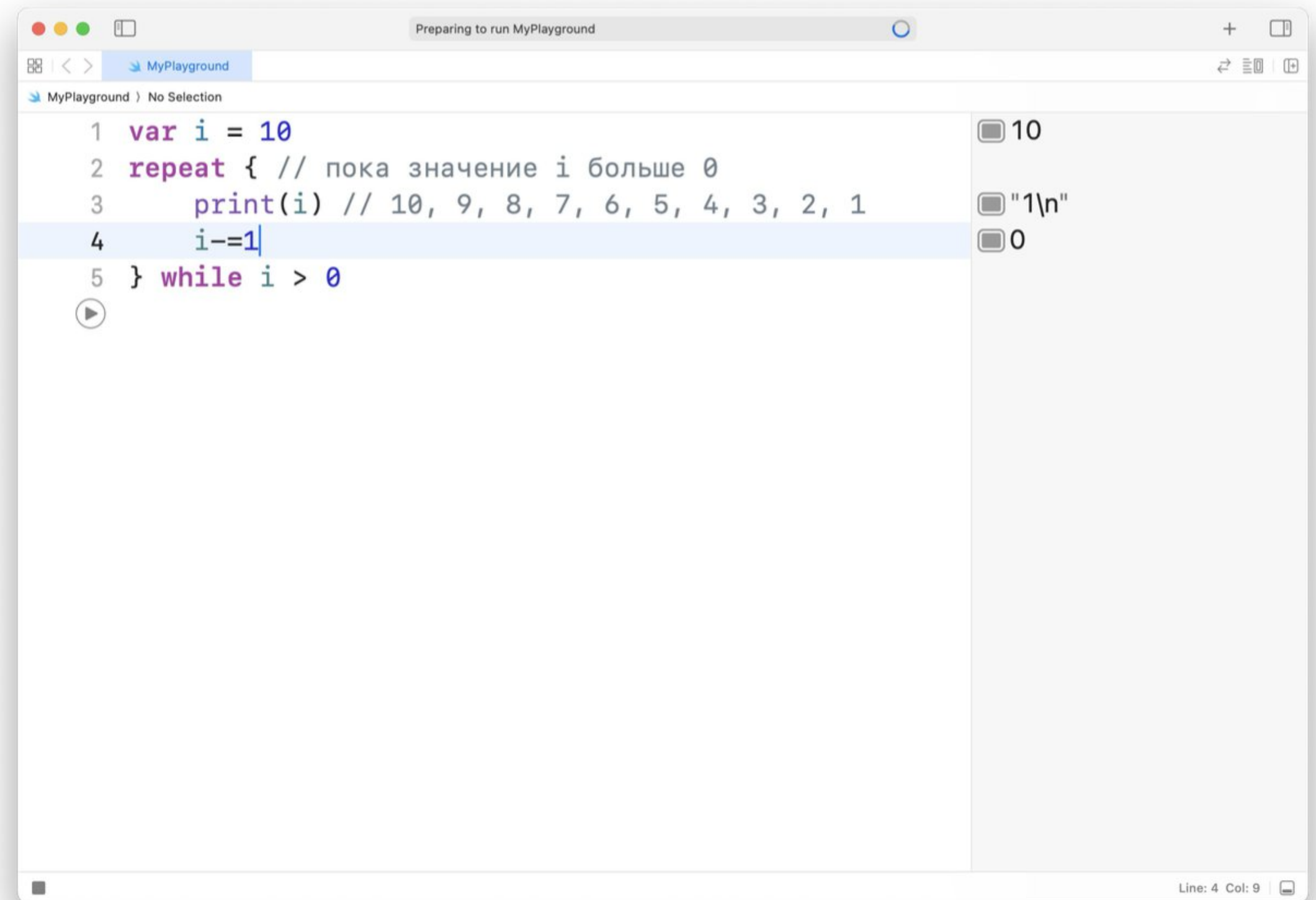
- 10
- "1\n"
- 0

The status bar at the bottom right indicates "Line: 4 Col: 9".

Цикл repeat-while. Цикл с постусловием

Цикл repeat-while сначала выполняет один раз тело цикла, и если некоторое условие возвращает true, то продолжает выполнение цикла. Он имеет следующую форму:

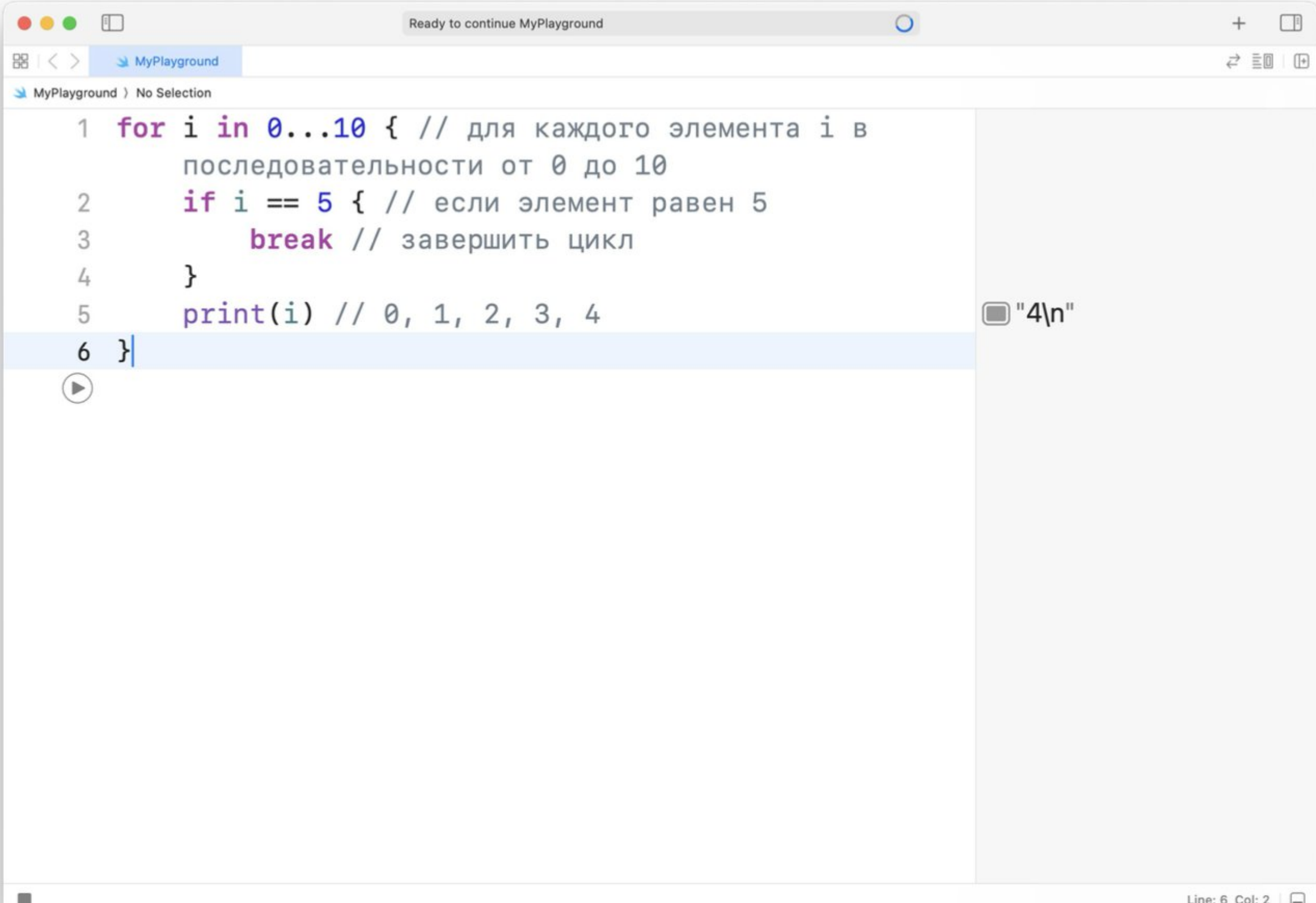
```
repeat {  
    // тело цикла  
} while условие
```



Операторы continue и break

Break

Иногда возникает ситуация, когда требуется выйти из цикла, не дожидаясь его завершения. В этом случае мы можем воспользоваться оператором break.



```
1 for i in 0...10 { // для каждого элемента i в
    последовательности от 0 до 10
2     if i == 5 { // если элемент равен 5
3         break // завершить цикл
4     }
5     print(i) // 0, 1, 2, 3, 4
6 }
```

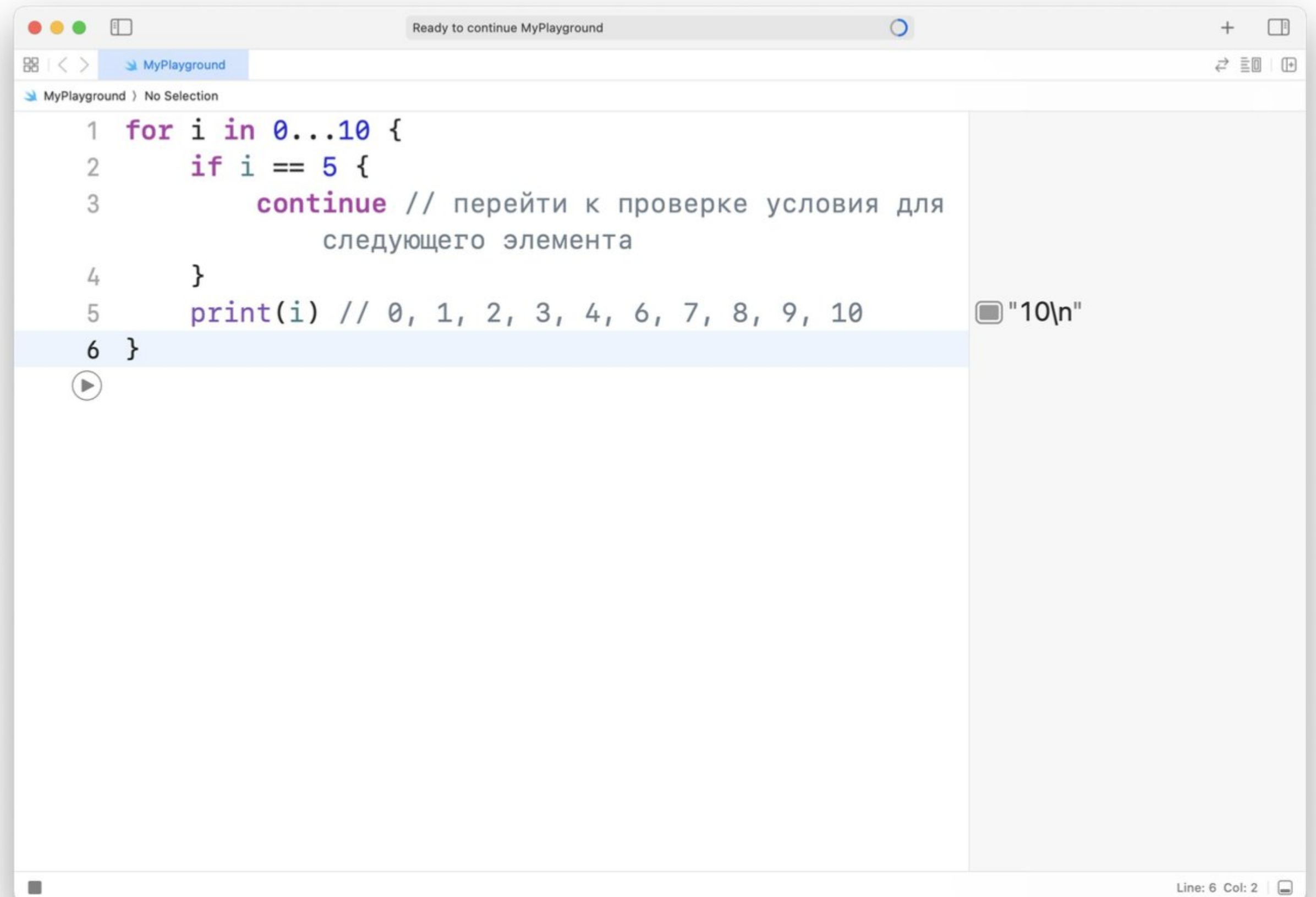
Output: "4\n"

Line: 6 Col: 2

Операторы continue и break

Continue

А что если мы хотим, чтобы при проверке цикл не завершился, а просто переходил к следующему элементу. Для этого мы можем воспользоваться оператором continue:



The screenshot shows a code editor window titled "Ready to continue MyPlayground". The code is as follows:

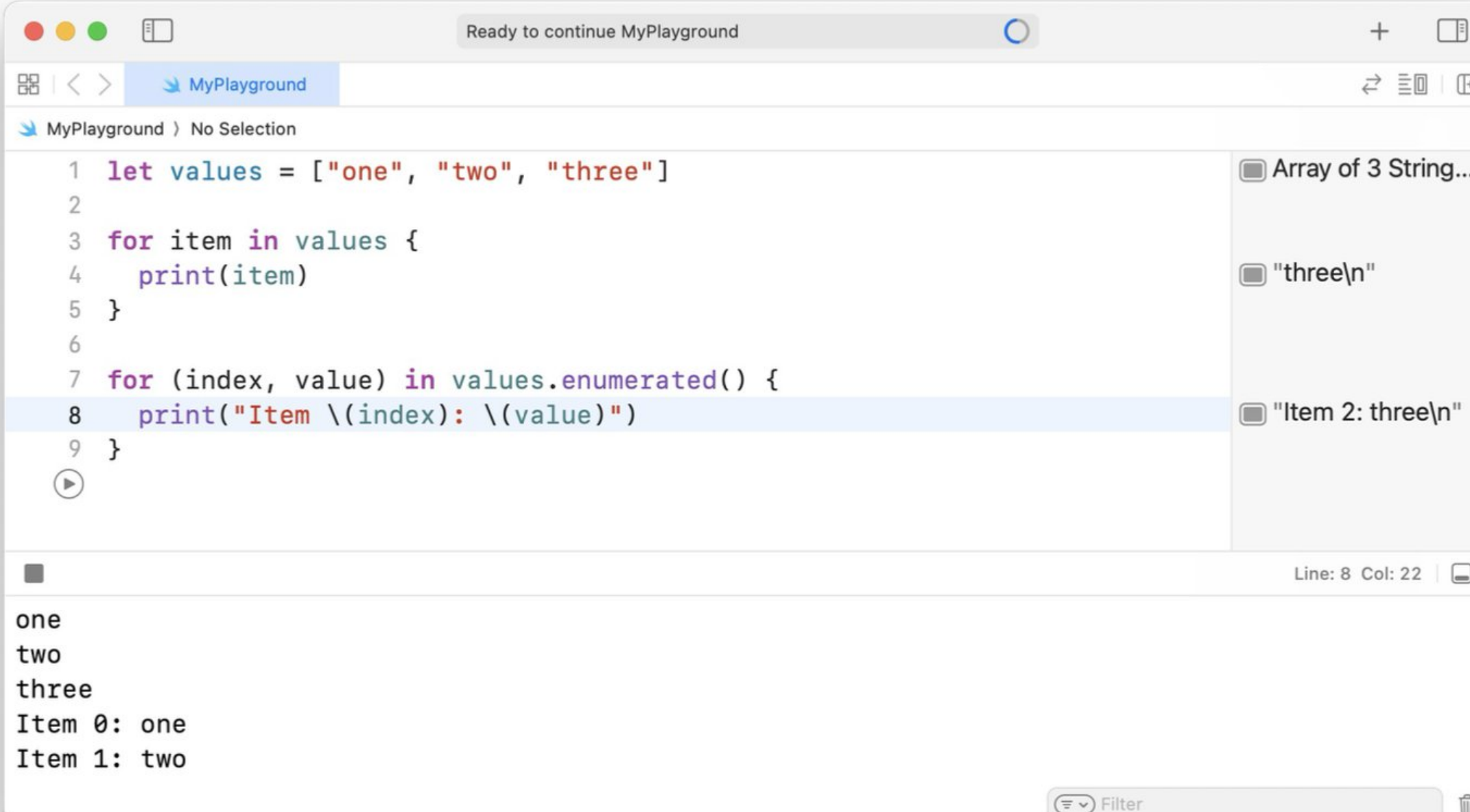
```
1 for i in 0...10 {  
2     if i == 5 {  
3         continue // перейти к проверке условия для  
                     следующего элемента  
4     }  
5     print(i) // 0, 1, 2, 3, 4, 6, 7, 8, 9, 10  
6 }
```

The code is written in a syntax-highlighted style. The line numbers 1 through 6 are on the left. The code is indented for the if block. A comment in Russian is provided for the continue statement. The print statement shows the output of the loop, skipping the value 5. The editor has a light blue background for the code area and a white background for the output area on the right. The output area shows the string "10\n". The status bar at the bottom right indicates "Line: 6 Col: 2".

Посмотрим на практике, обход массивов

Можно выполнить итерацию по всему набору значений внутри массива с помощью цикла `for-in`.

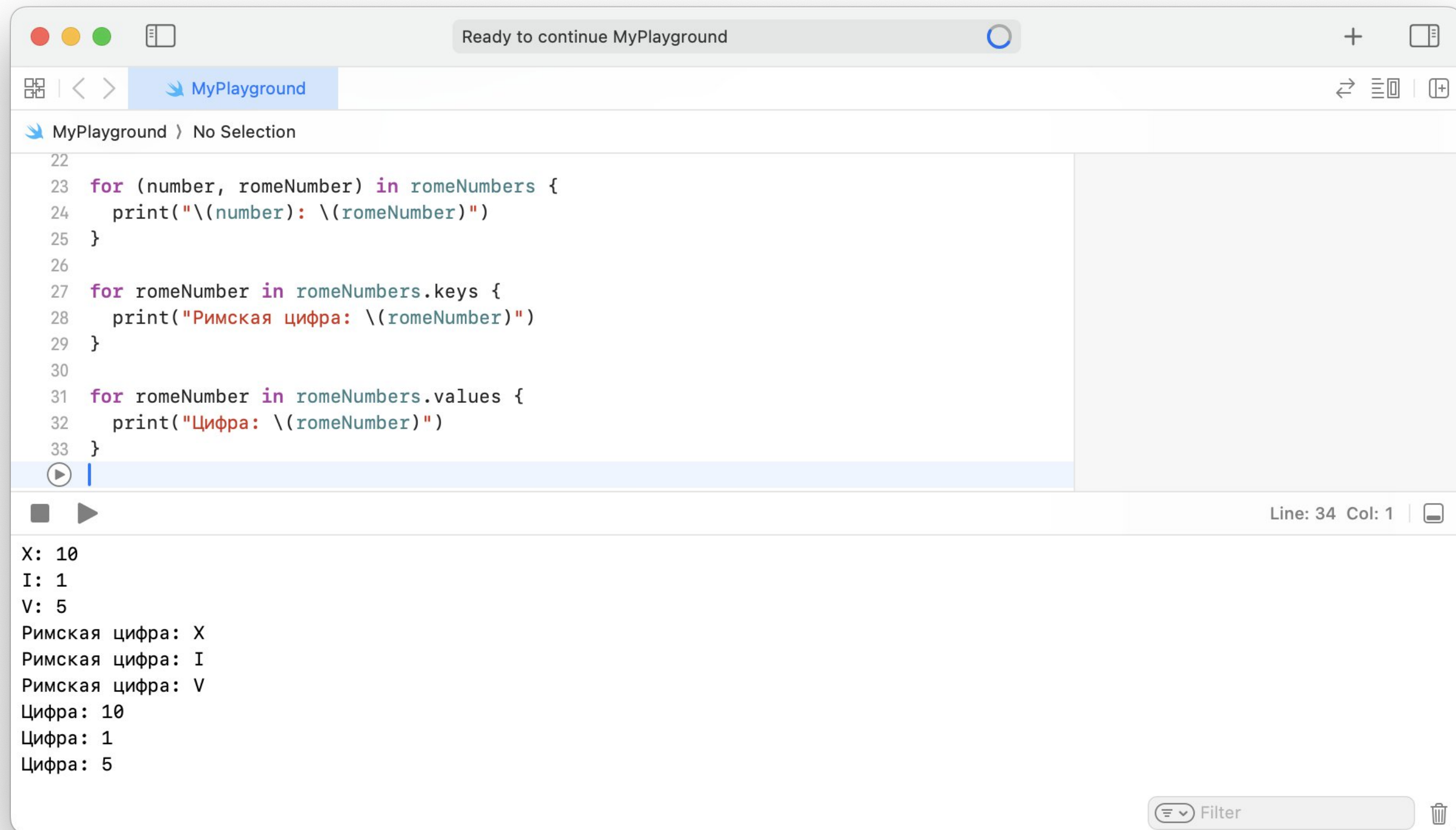
Если вам нужен целочисленный индекс каждого значения так же как и само значение, используйте вместо этого глобальную функцию `enumerated()` для итерации по массиву. Функция `enumerated()` возвращает кортеж для каждого элемента массива, собрав вместе индекс и значение для этого элемента.



```
1 let values = ["one", "two", "three"]
2
3 for item in values {
4   print(item)
5 }
6
7 for (index, value) in values.enumerated() {
8   print("Item \(index): \(value)")
9 }
```

one
two
three
Item 0: one
Item 1: two

Посмотрим на практике, обход словарей



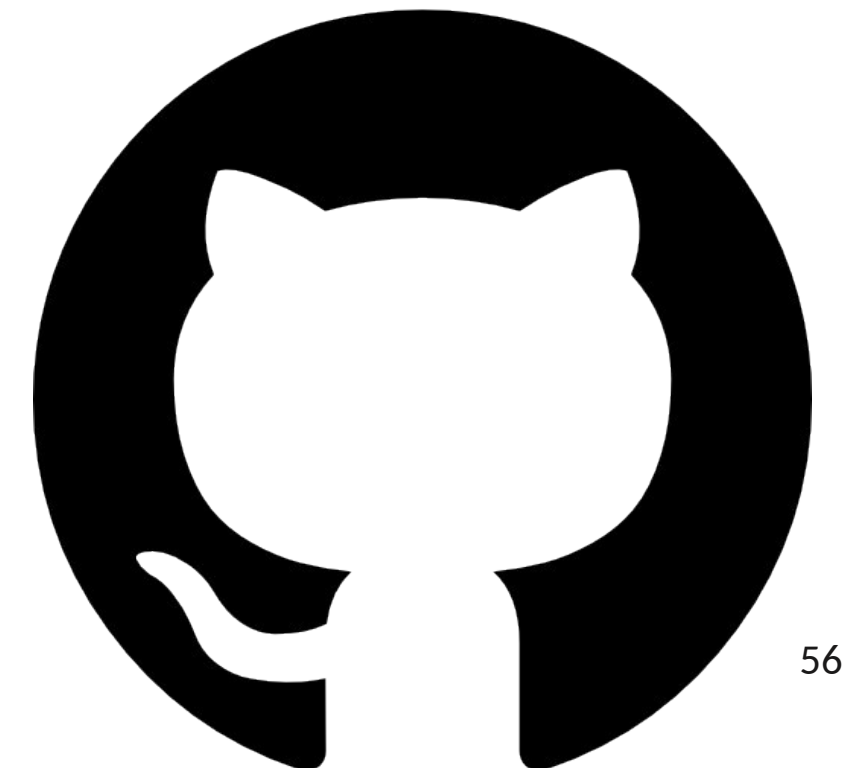
```
22
23 for (number, romeNumber) in romeNumbers {
24     print("\(number): \(romeNumber)")
25 }
26
27 for romeNumber in romeNumbers.keys {
28     print("Римская цифра: \(romeNumber)")
29 }
30
31 for romeNumber in romeNumbers.values {
32     print("Цифра: \(romeNumber)")
33 }
```

X: 10
I: 1
V: 5
Римская цифра: X
Римская цифра: I
Римская цифра: V
Цифра: 10
Цифра: 1
Цифра: 5

Live coding

Домашнее задание

- Текст задач найдете на edu
- Сделайте задания в Playground
 - Можно в одном, но тогда разделите задачи комментариями
- Сделайте ваши изменения в отдельной ветке
- PR в master (main) отправьте в форму



Спасибо!
Вопросы?

